

JAVA

How the JVM actually works under the hood — the illustrated field guide



with Cargo, your guide

Part of the Interview-Preparation illustrated series

Table of Contents

1. Stack vs. Heap
2. Generational Garbage Collection
3. HashMap: Buckets and Collisions
4. The String Pool
5. Checked vs. Unchecked Exceptions
6. synchronized: One Thread at a Time
7. Streams Are Lazy
8. Classloader Delegation
9. Cheat Sheet

This guide builds intuition for how the JVM actually behaves underneath your code -- the mental models that make debugging memory issues, concurrency bugs, and performance problems far more tractable.

CHAPTER 1

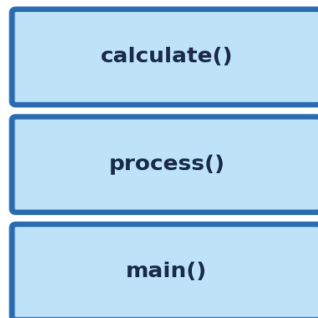
Stack vs. Heap

Every thread gets its own Stack -- a simple, fast, last-in-first-out structure holding local variables and method call frames. When a method returns, its frame is popped, and everything local to it disappears automatically.

The Heap is shared across every thread in the JVM. Every object you create with `new` lives here, regardless of which thread created it -- which is exactly why heap access needs synchronization when multiple threads touch the same object, while stack-local variables never do.

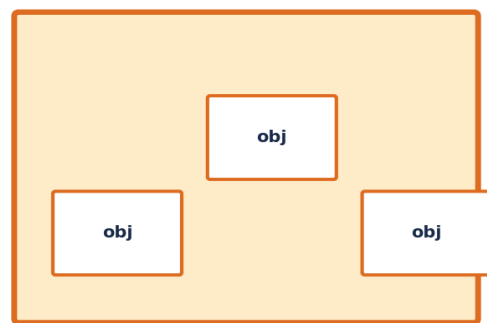
Stack vs. Heap

Stack (per thread)



*local variables,
method call frames*

Heap (shared)



all objects, garbage-collected

IN SHORT: *Stack = fast, per-thread, automatically cleaned up. Heap = shared, holds every object, cleaned up by the garbage collector instead of automatically on return.*

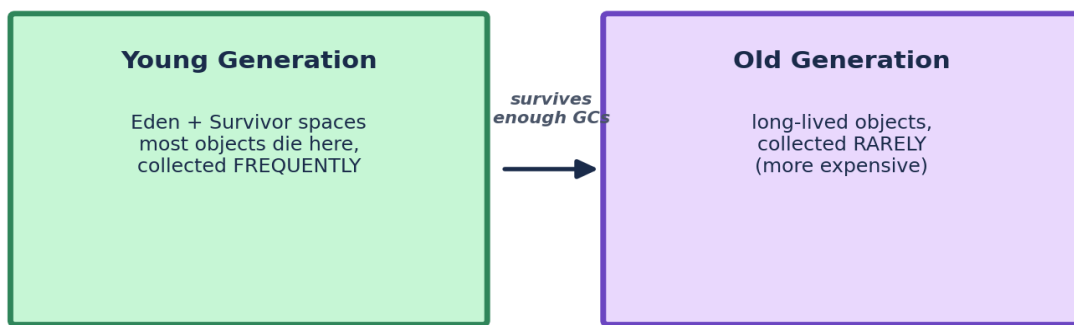
CHAPTER 2

Generational Garbage Collection

Most objects die young -- a temporary variable inside a loop, a short-lived request object. The JVM exploits this pattern by dividing the heap into generations rather than treating all objects identically.

New objects are allocated in the Young Generation, collected frequently and cheaply (most of what's there is already garbage by the time collection runs). Objects that survive several collections get promoted to the Old Generation, collected far less often since scanning it is comparatively expensive.

Generational Garbage Collection



IN SHORT: Frequent, cheap collection for young objects. Rare, expensive collection for the long-lived ones. This asymmetry is what makes Java's GC efficient in practice.

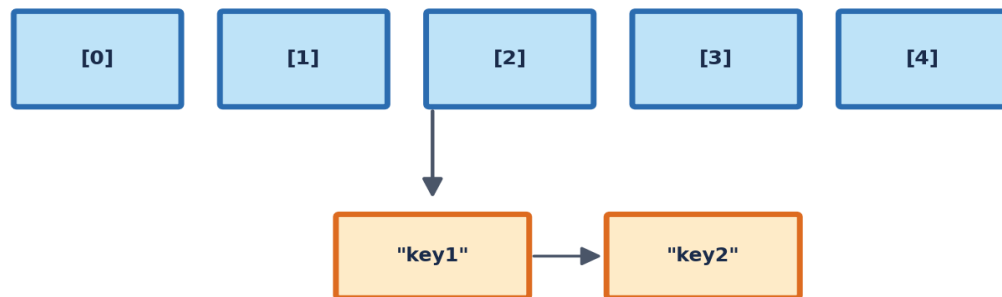
CHAPTER 3

HashMap: Buckets and Collisions

A HashMap stores entries in an array of buckets, with a key's hash code determining which bucket it lands in. Ideally each bucket holds at most one entry -- an $O(1)$ lookup.

When two different keys hash into the same bucket (a collision), Java chains them together as a linked list within that bucket. Since Java 8, if a single bucket's chain grows past 8 entries, it's converted into a red-black tree instead -- turning a worst-case $O(n)$ bucket scan into $O(\log n)$, protecting against both accidental and adversarial hash collisions.

HashMap: Buckets and Collisions



Same hash, different bucket slot -> chained (or, past 8 entries, a red-black tree).

IN SHORT: Good `hashCode()` implementations spread keys evenly across buckets. A bad one collapses your " $O(1)$ " map into a much slower linked list scan.

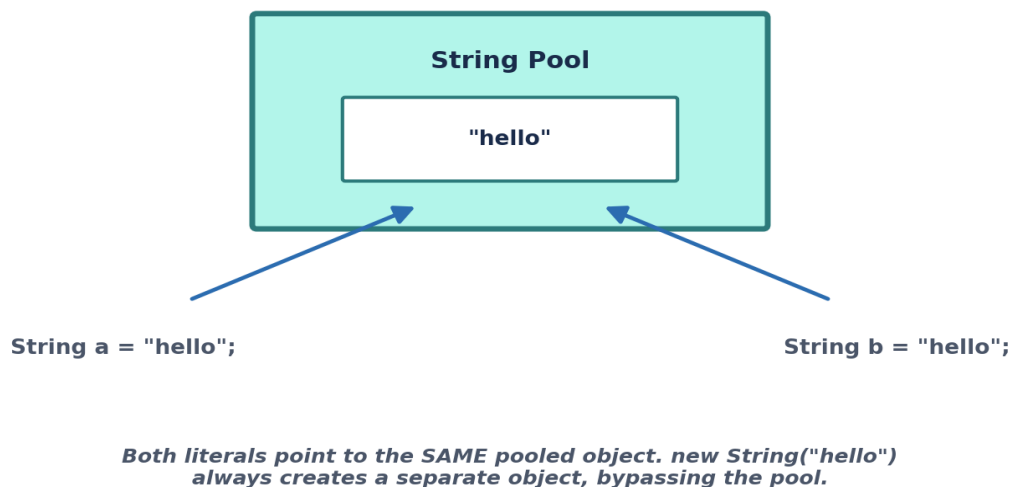
CHAPTER 4

The String Pool

String literals are interned automatically -- the JVM maintains a pool of unique string values, and any literal you write is checked against (and reused from) that pool rather than always allocating a fresh object.

This only applies to literals. `new String("hello")` deliberately bypasses the pool and creates a genuinely separate object on the heap, even though its content is identical -- which is exactly why `==` can behave unexpectedly on strings, and why you should always use `.equals()` to compare string content.

The String Pool



IN SHORT: Literals share pooled instances automatically. `new String(...)` never does. Always compare string content with `.equals()`, never `==`.

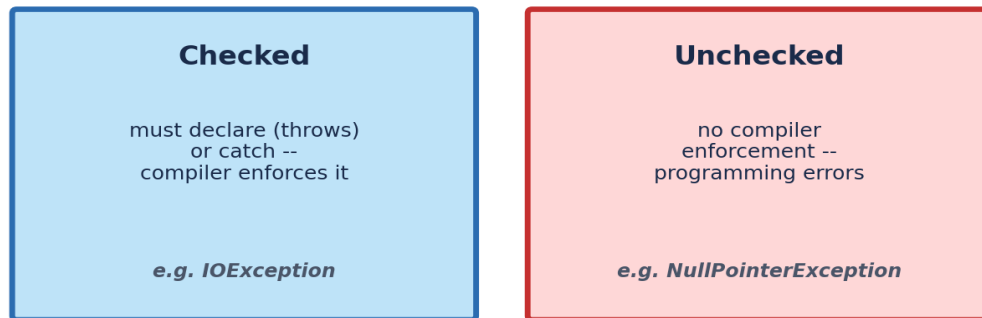
CHAPTER 5

Checked vs. Unchecked Exceptions

Checked exceptions (anything extending `Exception` but not `RuntimeException`) are enforced by the compiler -- you must either catch them or declare them in a `throws` clause. They represent conditions a caller is expected to anticipate and handle, like a file that might not exist.

Unchecked exceptions extend `RuntimeException` and face no such compiler enforcement -- they typically represent programming errors (a null reference, an invalid argument) that shouldn't need to be declared at every level of a call stack.

Checked vs. Unchecked Exceptions



IN SHORT: Checked = the compiler makes you handle it. Unchecked = a programming error the compiler trusts you to prevent, not recover from.

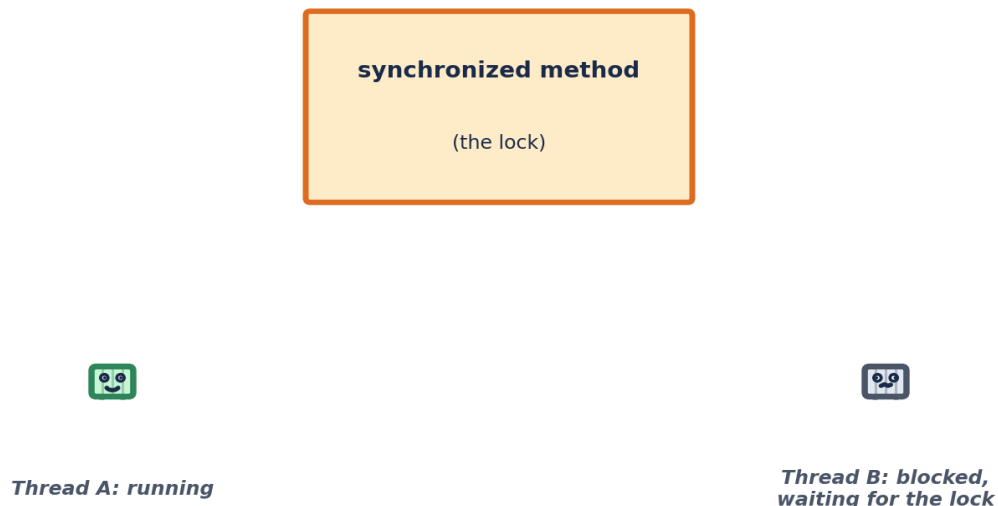
CHAPTER 6

synchronized: One Thread at a Time

A synchronized method or block requires a thread to acquire an intrinsic lock (the object's monitor) before entering. If another thread already holds that lock, any additional thread trying to enter simply blocks and waits.

This guarantees both mutual exclusion (only one thread executes the protected code at a time) and visibility (changes made inside are guaranteed visible to whichever thread acquires the lock next) -- the two properties needed for genuinely safe concurrent access to shared, mutable state.

synchronized: One Thread at a Time



IN SHORT: Only one thread can hold a given lock at a time. Every other thread requesting it simply waits its turn -- correctness at the cost of some concurrency.

CHAPTER 7

Streams Are Lazy

Intermediate stream operations like `filter()` and `map()` don't actually do any work when you call them -- they just build up a description of the pipeline. Nothing executes until a terminal operation, like `collect()` or `forEach()`, is called.

This laziness enables real optimizations: a short-circuiting terminal operation like `findFirst()` can stop processing the moment a match is found, without ever running the rest of the pipeline against the remaining elements.

Streams Are Lazy



Nothing runs until `collect()` (a terminal operation) is called -- `filter/map` are just a recipe until then.



IN SHORT: A stream pipeline is a recipe, not an action. Nothing runs until a terminal operation actually triggers execution.

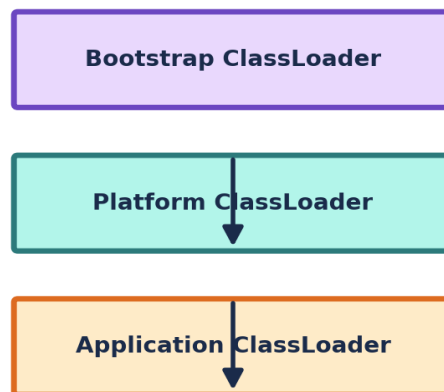
CHAPTER 8

Classloader Delegation

When a class needs to be loaded, Java doesn't just load it directly from wherever it finds it first -- it follows a strict parent-first delegation model across three classloaders: Bootstrap (core JDK classes), Platform, and Application (your own code).

A loading request is always delegated up to the parent first. Only if no parent loader already has a class by that name does the request actually fall through to be loaded by the current loader. This is precisely what prevents your own code from accidentally (or maliciously) overriding a core class like `java.lang.String`.

Classloader Delegation



A request always asks the PARENT first. Your own classes only load if no parent loader already has a class by that name.

IN SHORT: Requests always ask the parent first. This delegation model is what keeps core JDK classes safe from being silently shadowed by your own classpath.

CHAPTER 9

Cheat Sheet

The concepts you'll reach for constantly.

JAVA CHEAT SHEET



Memory

Stack: locals, method frames (per thread)

Heap: all objects (shared, GC-managed)

Young Gen: frequent, cheap GC

Old Gen: rare, expensive GC

Collections

ArrayList: fast random access

LinkedList: fast insert/remove at ends

HashMap: $O(1)$ avg, no order

TreeMap: $O(\log n)$, sorted

Concurrency

synchronized: one thread at a time

volatile: visibility, not atomicity

ExecutorService: manage thread pools

CompletableFuture: async pipelines

Exceptions

Checked: must declare or catch

Unchecked: RuntimeException family

try-with-resources: auto-close

Modern Java

var: local type inference

Records: immutable data carriers

Streams: declarative collection ops

Virtual threads: cheap concurrency